Question 1:

Suppose you have a dataset with classes as in table 1. Suppose you fit a k-nearest neighbours classifier with uniform weighting to this dataset with k = 1 (you can assume that the datapoint itself is included in the k nearest neighbours - it is its own nearest neighbour).

(i) What value of accuracy will you get? Briefly explain your answer.

The value of the accuracy will be 100%. This is because the values are already seen by the model and a rough decision boundary is formed for k = 1.

(ii) Now suppose you choose k equal to the number of datapoints. What value of accuracy will you get? Briefly explain your answer.

If k = Number of datapoints, the accuracy will be nearest to zero because each point of the dataset forms a class on its own and the class prediction is very likely not correct when it is close to 0.

(iii) Why are neither of these good values of k in general?

The principle of the k-nearest neighbours classifier is to look for groups which have almost the same characteristics. k = 1 or k = Number of datapoints are not good values of k in general because in these cases we do not group the data to have ideas on the characteristics of the groups, predict another value, and choose k that distinguishes us from groups with different characterization. A small value of k could lead to overfitting as well as a big value of k can lead to underfitting.

Class	Cats	Dogs	Cows
Num. Instances	50	30	20

Table 1: Animals

Question 2:

2(a)

In a naive Bayes classifier, the assumption is made that each feature is independent of each other feature. Give an example of a pair of features that you might encounter in the real world for which the assumption of independence doesn't hold, and a pair of features for which it does hold (more or less). Explain your answers.

- **Example of feature independence:** Gender-Age.
  There is no dependency relationship between age and sex, we can say whatever the age of a person x, their sex must not change, and this relationship remains true in the second sense.
- **Example of feature dependence:** Time of study - Score in a test.
  There is a strong relationship between study time and test score. Normally when a student spends most of their time studying, reviewing, and understanding the material properly, their marks are expected to be high. On the other hand, when they do not give sufficient time studying the unit materials, they tend to end up with a low grade.

2(b)

Suppose we have the following dataset describing three plant species:

	Species 1				
Altitude (100m)	2.20	2.04	1.95	1.79	1.91
Height (cm)	1.99	2.19	2.23	2.22	2.58

	Species 2				
Altitude (100m)	2.31	2.49	2.94	2.20	1.97
Height (cm)	2.53	2.20	1.88	2.68	2.93

	Species 3				
Altitude (100m)	1.02	1.54	0.82	2.93	0.76
Height (cm)	1.15	0.95	1.05	0.97	1.10

(i) Build a Gaussian naive Bayes classifier using this dataset and classify the point (2.1, 2.3). Show your working.

First of all, we need to calculate the mean and standard deviation for each feature in all classes. The following table shows the results:

Feature	Altitude		Height	
Class	Mean	SD	Mean	SD
Species 1	1.97	0.14	2.24	0.19
Species 2	2.38	0.33	2.44	0.37
Species 3	1.41	0.81	1.04	0.08

$\vec{x} = (2.1, 2.3)$

Species 1

P(C1) = $\frac{5}{30}$ = 0.166

P(X altitude | C1) = $\frac{1}{0.14\sqrt{2\pi}}$ exp (- $\frac{1}{2}$ ($\frac{2.1-1.97}{0.14}$)^2)

P(X height | C1) = $\frac{1}{0.19\sqrt{2\pi}}$ exp (- $\frac{1}{2}$ ($\frac{2.3-2.24}{0.19}$)^2)

Log(P(Ci)) + $\sum_{j=1}^{n}$ Log(P(xj |ci)) = Log(P(C1)) + Log P(X altitude | C1) + Log P(X height C1)

= Log(0.166) + Log(0.63669) + Log(0.96368)

= —0.78 — 0.20 — 0.02

= —1

Species 2

P(C2) = 0.166

P(X altitude | C2) = $\frac{1}{0.33\sqrt{2\pi}}$ exp (- $\frac{1}{2}$ ($\frac{2.1-2.38}{0.33}$)^2)

P(X height | C2) = $\frac{1}{0.37\sqrt{2\pi}}$ exp (- $\frac{1}{2}$ ($\frac{2.3-2.44}{0.37}$)^2)

Log(P(Ci)) + $\sum_{j=1}^{n}$ Log(P(xj |ci)) = Log(P(C2)) + Log P(X altitude | C2) + Log P(X height C2)

= Log(0.166) + Log(0.93398) + Log(0.99462)

= —0.78 — 0.03 — 0.002

= —0.812

Species 3

P(C3) = 0.166

P(X altitude | C3) = $\frac{1}{0.81\sqrt{2\pi}}$ exp (- $\frac{1}{2}$ ($\frac{2.1-1.41}{0.81}$)^2)

P(X height | C3) = $\frac{1}{0.08\sqrt{2\pi}}$ exp (- $\frac{1}{2}$ ($\frac{2.3-1.04}{0.08}$)^2)

Log(P(Ci)) + $\sum_{j=1}^{n}$ Log(P(xj |ci)) = Log(P(C3)) + Log P(X altitude | C3) + Log P(X height C3)

= Log(0.166) + Log(1.29299) + Log(2.80869)

= —0.78 + 0.11 + 0.45

= —0.22

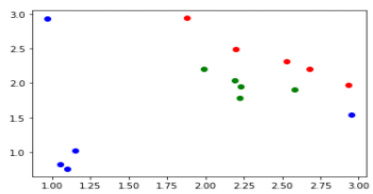
Since Class 3 (Species 3) maximises the quantity log(P (ci)) + $\sum_{j=1}^{n}$ log(P (xj |ci)), so we classify $\vec{x}$ = (2.1, 2.3) as belonging to class 3 (Species 3).

(ii) Do any of the classes fit the independence assumption? Do any of the classes violate the independence assumption? Explain your answer. (Hint: you may wish to plot the data)

The following scatter plot shows which classes fit the independence assumption:

```
import matplotlib.pyplot as plt
plt.scatter(df['hight'][:5],df['altitude'][:5],color = 'g')
plt.scatter(df['hight'][5:10],df['altitude'][5:10],color = 'r')
plt.scatter(df['hight'][10:15],df['altitude'][10:15],color = 'b')
plt.show
```

```
<function matplotlib.pyplot.show(close=None, block=None)>
```



- The first class (species 1) with the green color fit the independence assumption. All points are grouped.
- The third class (species 3) with the blue color violates the independence assumption. The points are not grouped.

2(c)

Would k-means do a good job at clustering this data? Explain why or why not.

We cannot use k-means with this data because k-means for clustering works with unlabelled data. And in this case, we notice that we have a labelled data.

Question 3:

Suppose we initialize a Gaussian mixture model with k components as follows:

- We run k-means on the dataset to get initial values for the mean of each component.
- We set the covariance matrix of each component to a small multiple of the identity matrix.
- We set the weight of each component to k1.

Explain why this initialization is equivalent to a k-means clustering. In particular, consider the membership of a datapoint in each cluster in k-means, and in each component in the mixture of Gaussians. Why are they (approximately) equal?

If we initialize a Gaussian mixture model with k components, we return to the knn algorithm steps which are as follows:
1- Choosing the number of clusters K
2- Initializing the vector μ_k that defines a central point of each cluster
3- Assigning each data point x to the closest cluster center
4- Recalculating central points μ_k for each cluster
5- Repeating 3–4 steps until central points stop moving
So, we come back to the algorithm written in the question, which applies the knn algorithm with its steps.

Question 4:

Your friend is coming to visit you in Bristol city, and you have decided to walk to the city museum. Unfortunately, you don't know your way to the museum that well, but you do know how to get to various different landmarks. You start planning a route based on the distances and heuristics given in the diagram in figure 1.

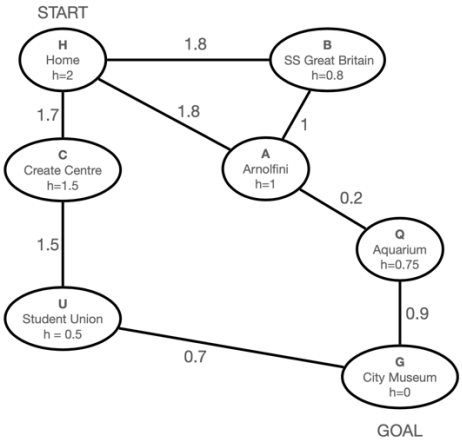


Figure 1: Bristol landmarks. Distances by road are given on the edges of the graph, and the heuristic values (h) are straight line distances between landmarks.

4(a)
What order will you expand the nodes in for the following (break any ties alphabetically)?

(i) Breadth-first search
H, A, B, C, Q, U, G

(ii) Depth-first search
H, A, B, Q, G

(iii) Best first greedy search
H, B, A, Q, G

(iv) Uniform cost search

H, C, A, B, Q, G

Explored	Frontier	Paths
H	A, B, C	HC (1.7), HA (1.8), HB (1.8)
HC	U, A, B	HA (1.8), HB (1.8), HCU (3.2)
HCA	Q, U, B	HB (1.8), HAQ (2.0), HCU (3.2)
HCAB	Q, U	HAQ (2.0), HBA (2.8), HCU (3.2)
HCABQ	U, G	HBA (2.8) – A has expanded, HAQG (2.9), HCU (3.2)
HCABQG	U	HCUG (2.2), HAQG (2.9)

Which, if any, of these techniques gives you the optimal route to the museum?

The best search technique that gives the optimal route to the museum is Uniform cost search.

4(b)

Is the heuristic:

1. Admissible?
2. Consistent?

Explain your answer.

Heuristic is admissible, the search can be done according to the length of the path, choosing the shortest path, and knowing that heuristics are methods that give results close to the minimum or the best results for that they are acceptable.

4(c)

Suppose the edge between the student union (node U) and the city museum (node G) now has a negative cost of -1 (because you particularly want to see the fountain on the way). Suppose you try to run uniform cost search on this graph (even though one edge is negative). Does uniform cost search still find the optimal path? If not, why not?

NB: do not simply answer that uniform cost search requires positive edge costs - please investigate what happens in this situation.

In this scenario the uniform cost search will not find the optimal path since the goal stage will be achieved with the path (H, A, Q, G) with a value of 2.9 when node Q is explored. At this point, the value of other explored path (H, C, U) will have a value of 3.2 and the unform cost search will not expand node U.

if we expand U then the path (H, C, U, G) will have a value of 2.2 which will be the optimal path in this case.

4(d)

Now suppose the edge between U and G has a cost of 0.7 as before. You discover that there is a ferry between SS Great Britain (B) and the Aquarium (Q), adding an edge between B and Q at

negative cost of -2 (your friend loves ferries). Suppose you try to run uniform cost search on this graph (even though one edge is negative). Does uniform cost search still find the optimal path? If not, why not?

NB: do not simply answer that uniform cost search requires positive edge costs - please investigate what happens in this situation.

If we add an edge between B and Q the new path (H, B, Q, G) will be the shortest path with a value of 0 in distance and it is not applicable. The algorithm will give us the path HBQG which is not applicable.

Question 5:

Consider the 8-puzzle problem given the initial state shown in figure 2. Assume that the space moves up if possible, left if up is not possible, down if neither up nor left are possible, otherwise right (i.e. the order of priority of the moves is up, left, down, right).

5(a)

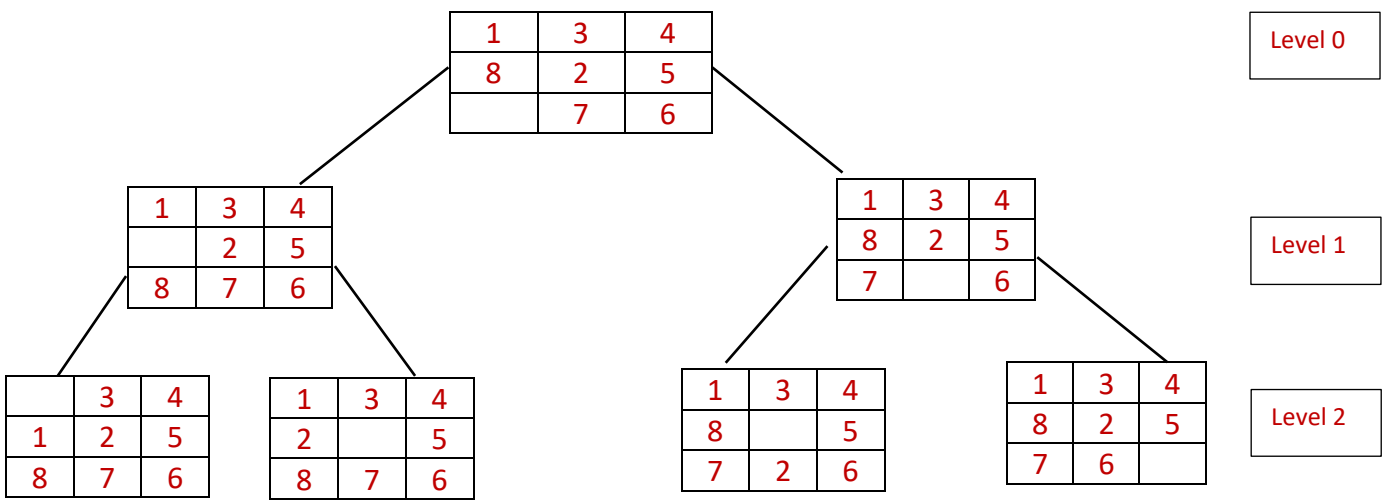
Using breadth-first search, show the search tree that would be built down to level two, assuming the root of the tree, i.e. the initial state, is level zero.

1	3	4
8	2	5
	7	6

1	2	3
8		4
7	6	5

Figure 2: 8 puzzle initial state (left) and goal state (right)

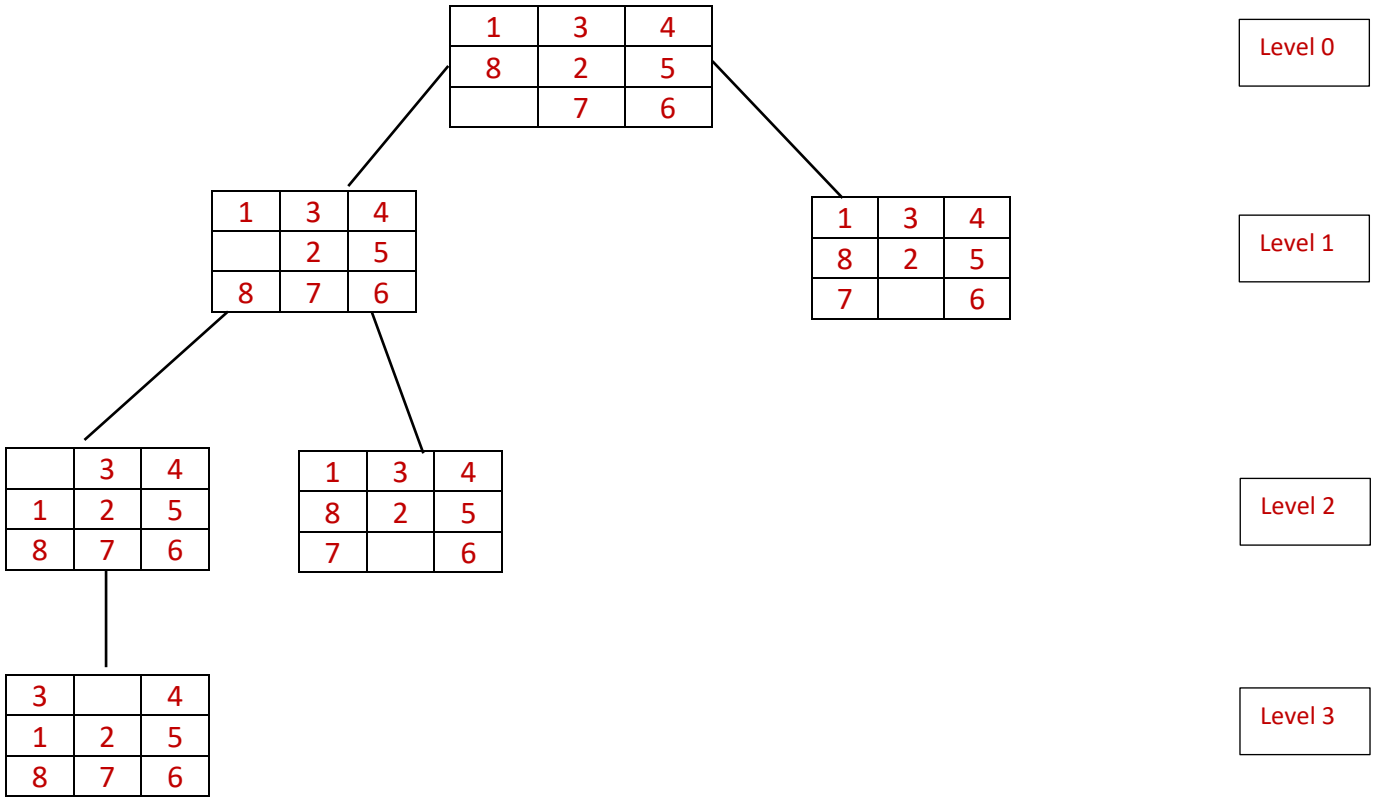
Using breadth-first search, the search tree that would be built down to level two, assuming the root of the tree:



5(b)

Figure 2: 8 puzzle initial state (left) and goal state (right)
Using depth-first search, show the search tree that would be built down to level three (stop after you have expanded one node that goes to level three).

Using depth-first search, the search tree that would be built down to level three:

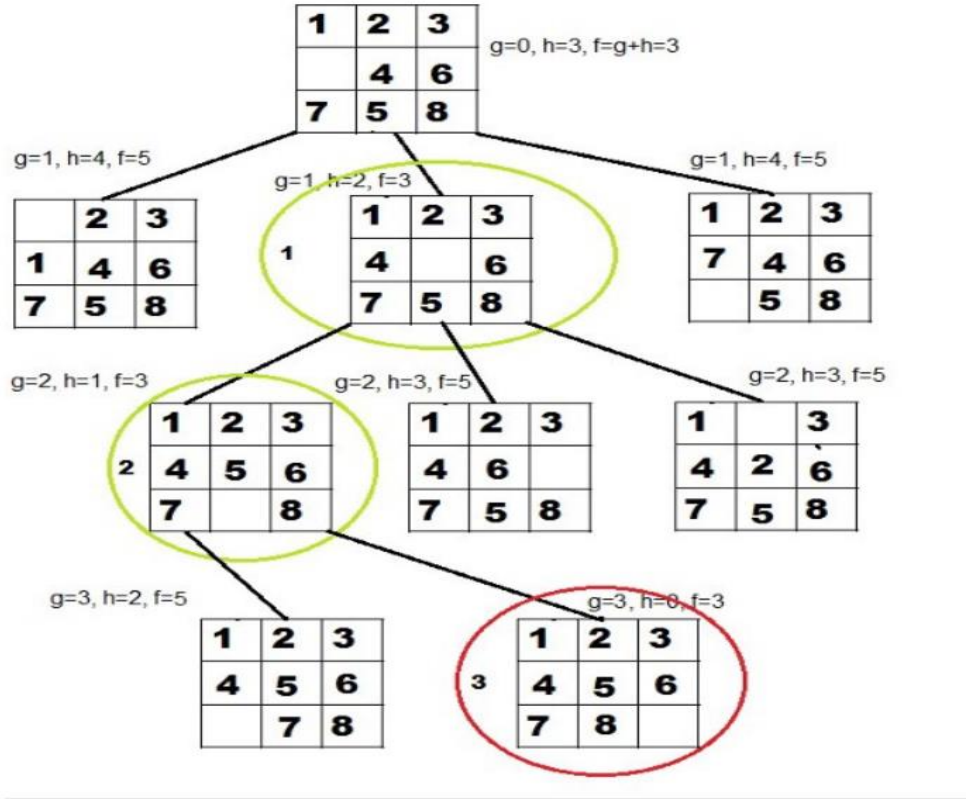


5(c)

Choose a heuristic function for the 8 puzzle and solve this instance using A* search.

How A* search solves the 8-Puzzle problem.

We first move the empty space in all the possible directions in the start state and calculate the f-score for each state. This is called expanding the current state.
After expanding the current state, it is pushed into the closed list and the newly generated states are pushed into the open list. A state with the least f-score is selected and expanded again. This process continues until the goal state occurs as the current state. Basically, here we are providing the algorithm a measure to choose its actions. The algorithm chooses the best possible action and proceeds in that path.
This solves the issue of generating redundant child states, as the algorithm will expand the node with the least f-score.



5(d)

On this problem, will breadth-first search find the optimal solution? Explain why, or why not.

In this problem the breadth-first search algorithm will find the optimal solution because it chooses first the best solution which converges towards the optimal solution.

Question 6:

You are on your way home, and you remember that you left a slice of cake out on the table. You wonder if it will be eaten (C): if your flatmate is home (F) then they may have eaten your cake. Also, the dog often gets loose (D), and if so, the dog may have eaten your cake. If the dog does get loose, it is likely to have dug a hole in the garden (H). This can be formulated as a Bayesian network as in figure 3. The probability tables are given below.

- C is a random variable specifying whether your cake is eaten, c indicates that is is eaten, ¬c that it is not eaten.
- D is a random variable specifying whether the dog is loose, d indicates the dog is loose, ¬d indicates it is not loose.+
-
- F is a random variable specifying whether your flatmate is home, f indicates your flatmate is home, ¬f indicates they are not home.

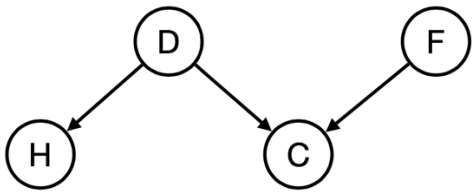


Figure 3: Bayesian network for the cake situation

- H is a random variable specifying whether there is a hole in the garden, h indicates there is a hole in the garden, ¬h indicates there is not a hole in the garden.

D	$P(D)$
d	0.3
$\neg d$	0.7

F	$P(F)$
f	0.4
$\neg f$	0.6

D	H	$P(H\|D)$
d	h	0.5
d	$\neg h$	0.5
$\neg d$	h	0.1
$\neg d$	$\neg h$	0.9

D	F	C	$P(C\|D,F)$
d	f	c	1
d	f	$\neg c$	0
d	$\neg f$	c	0.8
d	$\neg f$	$\neg c$	0.2
$\neg d$	f	c	0.9
$\neg d$	f	$\neg c$	0.1
$\neg d$	$\neg f$	c	0.1
$\neg d$	$\neg f$	$\neg c$	0.9

(i) What is the probability that there is a hole in the garden?

The probability that there is a hole in the garden:
P(H) =P(H, D) + P(H , D')
=P(H|D) * P(D) + P(H|D')*P(D')
= 0.5 * 0.3 + 0.1 * 0.7
= 0.22

(ii) What is the probability that the dog is loose, given that there is a hole in the garden?

The probability that the dog is loose, given that there is a hole in the garden:
P (D I H) = (P(H I D)*P(D))/P(H)
= (0.5*0.3)/0.22
= 0.68

(iii) What is the probability that the dog is loose, given that your cake is eaten and your flatmate is home?

The probability that the dog is loose, given that your cake is eaten, and your flatmate is home:
P(D I C,F) = P(D,C,F)/P(C,F)
= P(D)*P(F)*P(C I F, D))/(P(D)*P(F)*P(C | F, D) + P(non D)*P(F)*P(C |nonD, F)
= 0.3 * 0.4 * 1 / (0.3 * 0.4 * 1) + (0.7 * 0.4 * 0.9)
= 0.32

(iv) What is the probability that your flatmate is home, given that your cake is eaten and there is a hole in the garden?

The probability that your flatmate is home, given that your cake is eaten and there is a hole in the garden:
P(F I C,H) = (P(C)*P(H I C)*P(F I H))/(P(C)*P(H I C))
= P(F)
= 0.4

Question 7:

Consider an agent in a 2x3 gridworld. The rewards for the gridworld are given in table 2

2	-0.5	10 G
-0.5	-0.5	-0.5

Table 2: Rewards in the 2x3 gridworld

The top right cell (3, 2), marked G, is the goal state. Once the goal state is reached, the game is over. The agent starts in the bottom left cell (1, 1).

The transition model says that when the agent moves, it will go in the intended direction 80% of the time, and at right angles to the intended direction the rest of the time. So if the agent moves up, it will go up with probability 0.8, left with probability 0.1, and right with probability 0.1. Suppose the discount factor is 0.9.

7(a)
We will use value iteration to calculate the utility of each state. Assume that at iteration 0, all cells have utility 0. After one iteration, the utility of each state is equal to its reward, as in table 3. Explain why this is the case.

2	-0.5	10 G
-0.5	-0.5	-0.5

Table 3: Utilities at iteration 1

After one iteration, the utility of each state is going to be equal to its reward. This is because the utility at iteration 0 is equal to 0. Hence, after one round of value iteration, the utility of each state will be like the following according to bellman equation:

$$U_1(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s'|a,s) U_0(s') \quad = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s'|a,s) * 0 \quad (1)$$

$$= R(s) + 0 = R(s) \quad (2)$$

- U1(1, 1) = R(1, 1) + 0.9(0.8 × 0 + 0.1 × 0 + 0.1 × 0)
  = —0.5 + 0
  = —0.5
- U1(1, 2) = R(1, 2) + 0.9(0.8 × 0 + 0.1 × 0 + 0.1 × 0)
  = 2 + 0
  = 2

- U1(2, 1) = R(2, 1) + 0.9(0.8 × 0 + 0.1 × 0 + 0.1 × 0)
  = —0.5 + 0
  = —0.5
- U1(2, 2) = R(2, 2) + 0.9(0.8 × 0 + 0.1 × 0 + 0.1 × 0)
  = —0.5 + 0
  = —0.5
- U2(3, 1) = R(3, 1) + —0.5
  = —0.5 + 0
  = —0.5

7(b)

Use value iteration to calculate the utility of each cell in the gridworld after two and three iterations, writing these utilities down each time. What is the optimal policy according to the utilities after three iterations? Does the agent reach the goal?

Rewards

2	-0.5	10 G
-0.5	-0.5	-0.5

Utilities at iteration 1

2	-0.5	10 G
-0.5	-0.5	-0.5

Utilities at iteration 2

3.58	6.61	10 G
0.85	-0.95	6.61

Utilities at iteration 3

8.74	14.32	10 G
3.42	4.48	14.32

- U2(1, 1) = R(1, 1) + 0.9(0.8 × 2 + 0.1 × —0.5 + 0.1 × —0.5)
  = —0.5 + 1.35
  = 0.85 (Up is the best action)
- U2(1, 2) = R(1, 2) + 0.9(0.8 × 2 + 0.1 × 2 + 0.1 × —0.5)
  = 2 + 1.575
  = 3.58 (Left is the best action)
- U2(2, 1) = R(2, 1) + 0.9(0.8 × —0.5 + 0.1 × —0.5 + 0.1 × —0.5)
  = —0.5 —0.45
  = —0.95 (All actions are the same)
- U2(2, 2) = R(2, 2) + 0.9(0.8 × 10 + 0.1 × —0.5 + 0.1 × —0.5)
  = —0.5 + 7.11
  = 6.61. (Up is the best action)
- U2(3, 1) = R(3, 1) + 0.9(0.8 × 10 + 0.1 × —0.5 + 0.1 × —0.5)
  = —0.5 + 7.11
  = 6.61 (Up is the best action)
- U3(1, 1) = R(1, 1) + 0.9(0.8 × 3.58 + 0.1 × 0.85 + 0.1 × —0.95)
  = 0.85 + 2.5686
  = 3.42 (Up is the best action)

- U3(1, 2) = R(1, 2) + 0.9(0.8 × 6.61 + 0.1 × 3.58 + 0.1 × 0.85)
  = 3.58 + 5.1579
  = 8.74 (Up is the best action)
- U3(2, 1) = R(2, 1) + 0.9(0.8 × 6.61 + 0.1 × 0.85 + 0.1 × 6.61)
  = —0.95 + 5.4306
  = 4.48 (Up or Right are the best actions)
- U3(2, 2) = R(2, 2) + 0.9(0.8 × 10 + 0.1 × 6.61 + 0.1 × —0.95)
  = 6.61 + 7.7094
  = 14.32 (Up is the best action)
- U3(3, 1) = R(3, 1) + 0.9(0.8 × 10 + 0.1 × —0.95 + 0.1 × 6.61)
  = 6.61 + 7.7094
  = 14.32 (Up is the best action)

Optimal Policy according to the utilities after two iterations

Up	Up	X
Up	Up	Up

Optimal Policy according to the utilities after three iterations

Up	Left	X
Up	Up	Right

The agent does not reach the goal. It appears that the optimal path for the agent at iteration 3 is to go left and stay in its square (2, 2) = 14.32 as the goal state has a lower expected utility = 10.

7(c)

After convergence, the states have the utilities given in table 4. What is the optimal policy

17.34	14.38	10.0 G
14.38	12.25	10.13

Table 4: Utilities at iteration 90

according to the utilities after convergence? Does the agent reach the goal?

Optimal Policy according to the utilities after convergence

Up	Left	X
Up	Up	Left

The agent does not reach the goal. It appears that the optimal path for the agent at iteration 90 is to go up and stay in its square (1, 2) = 17.34 as it has the highest utility.

7(d)

Suppose the discount factor is reduced to 0.01. What will the optimal policy be? Is the optimal policy different than when the discount factor is 0.9? Explain why, or why not.

Calculating utilities of the states after two and three iterations with a discount factor = 0.01

- U2(1, 1) = R(1, 1) + 0.01(0.8 × 2 + 0.1 × —0.5 + 0.1 × —0.5)
  = —0.5 + 0.015
  = —0.49

- U2(1, 2) = R(1, 2) + 0.01(0.8 × 2 + 0.1 × 2 + 0.1 × —0.5)
  = 2 + 0.0175
  = 2.02
- U2(2, 1) = R(2, 1) + 0.01(0.8 × —0.5 + 0.1 × —0.5 + 0.1 × —0.5)
  = —0.5 —0.005
  = —0.50
- U2(2, 2) = R(2, 2) + 0.01(0.8 × 10 + 0.1 × —0.5 + 0.1 × —0.5)
  = —0.5 + 0.079
  = —0.42
- U2(3, 1) = R(3, 1) + 0.01(0.8 × 10 + 0.1 × —0.5 + 0.1 × —0.5)
  = —0.5 + 0.079
  = —0.42
- U3(1, 1) = R(1, 1) + 0.01(0.8 × 2.02 + 0.1 × —0.49 + 0.1 × —0.50)
  = —0.49 + 0.01517
  = —0.47
- U3(1, 2) = R(1, 2) + 0.01(0.8 × —0.42 + 0.1 × 2.02 + 0.1 × —0.49)
  = 2.02 + —0.00183
  = 2.01
- U3(2, 1) = R(2, 1) + 0.01(0.8 × —0.42 + 0.1 × —0.49 + 0.1 × —0.42)
  = —0.50 + —0.00427
  = —0.50
- U3(2, 2) = R(2, 2) + 0.01(0.8 × 10 + 0.1 × —0.42 + 0.1 × —0.50)
  = —0.42 + 0.07908
  = —0.34
- U3(3, 1) = R(3, 1) + 0.01(0.8 × 10 + 0.1 × —0.50 + 0.1 × —0.42)
  = —0.42 + 0.07908
  = —0.34

Utilities at iteration 2

2.02	-0.42	10 G
-0.49	-0.50	-0.42

Utilities at iteration 3

2.01	-0.34	10 G
-0.47	-0.50	-0.34

Optimal Policy according to the utilities after two iterations

Left	Up	X
Up	Up	Up

Optimal Policy according to the utilities after three iterations

Left	Up	X
Up	Up	Up

We can notice that the optimal policy is quite the same after two and three iterations when the discount factor equals 0.01 whereas the optimal policy changes for each iteration when the discount factor equals 0.9 until they convergence at iteration 90. Therefore, when the discount factor is small like 0.01 the utilities values will not change that much hence the optimal policy will stay the same and they will come to convergence at early iterations.